

Dhaka International University

Report Name

Testing and Validation Report of an Inventory Management System

A REPORT SUBMITTED TO

Md.Didar Ahmed

Lecture Of Dhaka International University

Software Testing Quality Assurance Lab - 0613-308

BY

Group - 4

Shazidul Islam Suvo Roll-23

Rahimur Rahman Rahim Roll-25

Abul Kalam Azad Kiron Roll-33

Tarik Jamil Roll-35

Date of Submission 29 April 2026

Table of Contents

Login Page	3-5
Registration Page	5-9
Customer Page	9-14
Category Page	14-19
Product Page	20-26
Create Sale Page	26-31
Invoice Page	31-34
Report Page	35-37

Login Page

Test Case: Login with Invalid Password

Test Case ID: TC_LOGIN_01

Test Scenario: Attempt login without entering password

Pre-condition: User is on the Sign In page

Test Steps:

1. Enter valid email in User Email field.
2. Leave the User Password field blank.
3. Click on NEXT button

Test Data:

- User Email: suvoislam1234@gmail.com
- User Password: blank

Expected Result:

- System should display an error message
- User should remain on the Sign In page

Actual Result:

System logged in successfully without showing any error message

Status: Fail

As shown in Figure 1, the system displays a validation message when the password field is left empty.

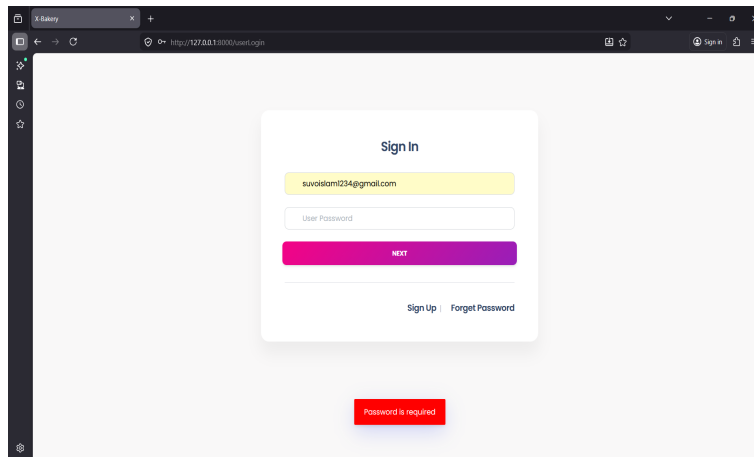


Figure 1: Password Required Message

Test Case: Login with Valid Credentials

Test Case ID: TC.LOGIN.02

Test Scenario: Login with valid email and password

Pre-condition: User is on the Sign In page

Test Steps:

1. Enter valid email in User Email field
2. Enter valid password in User Password field
3. Click on NEXT button

Test Data:

- User Email: admin@gmail.com
- User Password: 12345

Expected Result:

- System should successfully log in
- User should be redirected to the dashboard page

Actual Result:

User successfully logged in and redirected to dashboard page

Status: Pass

Figure 2 illustrates the user sign-in page where users can enter their email and password.

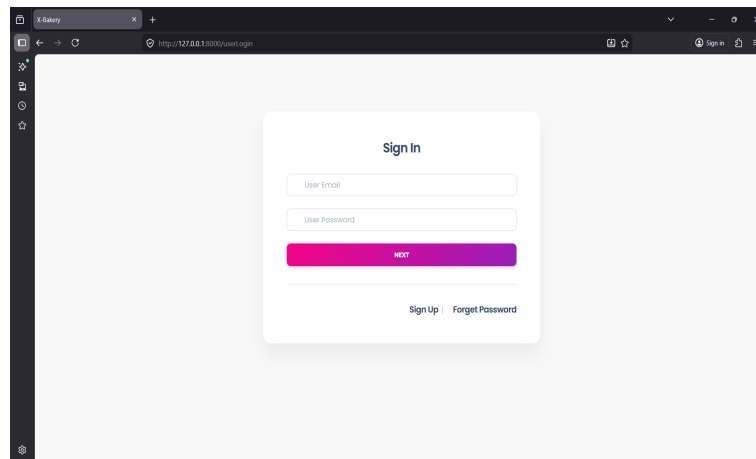


Figure 2: User Sign In Page Interface

Testing Approach for Login Module

The Login module testing is performed using **Black Box Testing**. In this approach, the internal structure or implementation of the system is not considered. Instead, the system is tested based on inputs and expected outputs.

In the above test cases, different input conditions such as valid credentials and missing password are provided to the system. The system's behavior is then observed, such as whether it allows access or shows an error message.

Since the testing focuses only on user interaction and system response without analyzing the internal code logic, this module follows the Black Box Testing approach.

Registration page**Test Case: User Registration with Valid Data**

Test Case ID: TC_REG_01

Test Scenario: Register a new user with valid information

Pre-condition: User is on the Sign Up page

Test Steps:

1. Enter valid email in Email Address field
2. Enter valid first name in First Name field
3. Enter valid last name in Last Name field
4. Enter valid mobile number in Mobile Number field
5. Enter valid password in Password field
6. Click on COMPLETE button

Test Data:

- Email Address: user@gmail.com
- First Name: Walter
- Last Name: White
- Mobile: 01721227088
- Password: Sdf123@!

Expected Result:

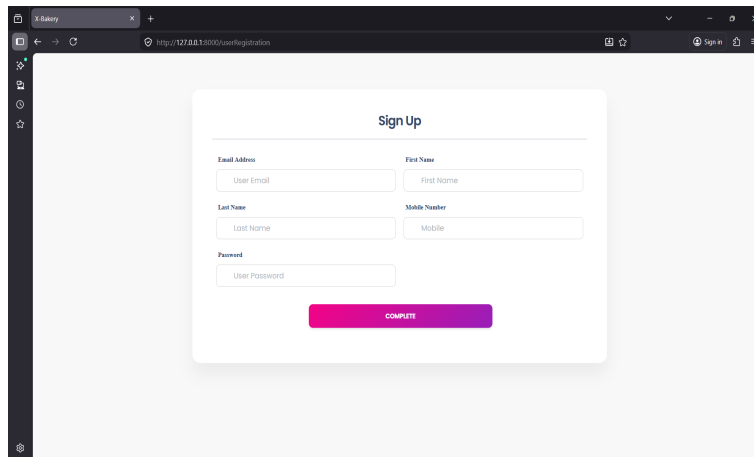
- User should be successfully registered
- System should redirect to dashboard page

Actual Result:

User successfully registered and redirected to login page

Status: Pass

Figure 3 illustrates the user registration page where new users can enter their personal details such as email, name, mobile number, and password.



The image shows a web browser window with a 'Sign Up' form. The form is titled 'Sign Up' and contains five input fields: 'Email Address' (labeled 'User Email'), 'First Name', 'Last Name', 'Mobile Number' (labeled 'Mobile'), and 'Password' (labeled 'User Password'). A pink button labeled 'COMPLETE' is positioned below the fields. The browser's address bar displays 'http://127.0.0.1:5000/UserRegistration'.

Figure 3: User Registration Page Interface

Test Case: User Registration with Blank Fields

Test Case ID: TC_REG_02

Test Scenario: Register a new user with missing information

Pre-condition: User is on the Sign Up page

Test Steps:

1. Leave Email Address field empty
2. Leave First Name field empty
3. Leave Last Name field empty
4. Leave Mobile Number field empty
5. Leave Password field empty
6. Click on COMPLETE button

Test Data:

- Email: blank
- First Name: blank
- Last Name: blank

- Mobile: blank
- Password: blank

Expected Result:

- System should display validation error messages for required fields
- User should not be registered

Actual Result:

System Registration successfully without showing any error message

Status: Fail

As shown in Figure 4, the system displays validation messages when required fields are left blank.

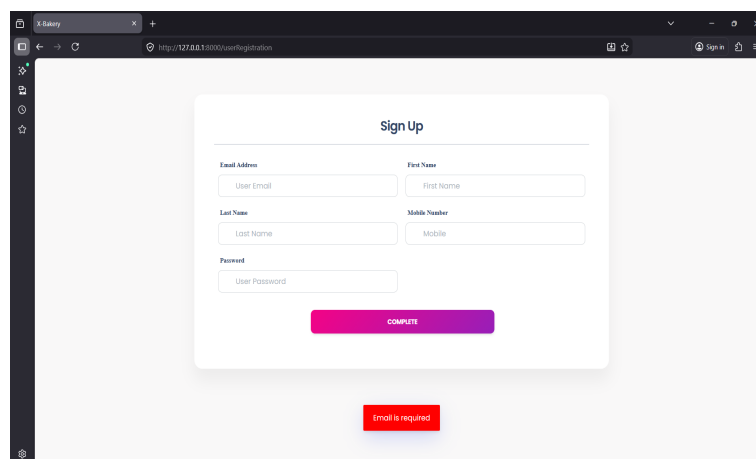


Figure 4: Required Fields Error Message

Testing Approach for Registration Module

The Registration module is also tested using **Black Box Testing**. The test cases are designed based on functional requirements, where various input combinations are provided without considering how the system processes the data internally.

For example, valid user data is used to verify successful registration, and blank fields are

tested to check whether proper validation messages are displayed.

The testing process evaluates only the correctness of output based on given input conditions. As no internal code structure or algorithm is analyzed, this module is categorized under Black Box Testing.

Customer Page

Test Case: Create Customer with Valid Data

Test Case ID: TC_CUST_01

Test Scenario: Add a new customer with valid information

Pre-condition: User is on Customer page

Test Steps:

1. Click on CREATE button
2. Enter valid Customer Name
3. Enter valid Customer Email
4. Enter valid Customer Mobile
5. Click on SAVE button

Test Data:

- Customer Name: Walter
- Customer Email: user@gmail.com
- Customer Mobile: 01721227088

Expected Result:

- Customer should be successfully added

- New customer should appear in the customer list

Actual Result:

Customer added successfully and displayed in the list

Status: Pass

Figure 5 illustrates the customer creation form where users can input customer name, email, and mobile number to add a new customer.

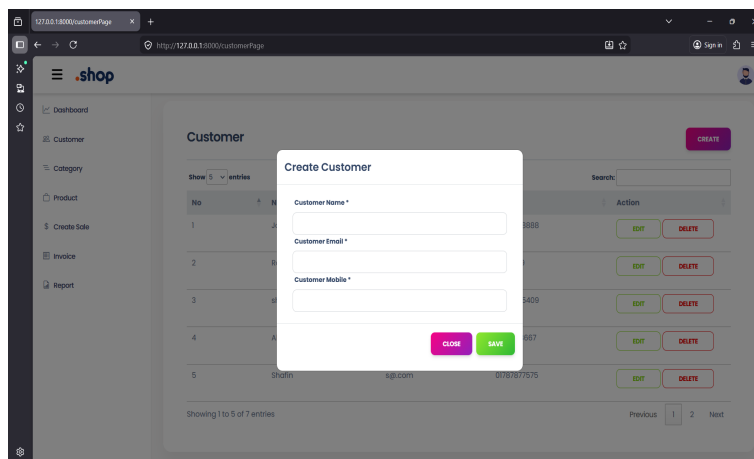


Figure 5: Customer Creation Form Interface

Test Case: Create Customer with Blank Data

Test Case ID: TC_CUST_02

Test Scenario: Add a new customer without entering any information

Pre-condition: User is on Customer page

Test Steps:

1. Click on CREATE button
2. Leave Customer Name field empty
3. Leave Customer Email field empty
4. Leave Customer Mobile field empty

5. Click on SAVE button

Test Data:

- Customer Name: blank
- Customer Email: blank
- Customer Mobile: blank

Expected Result:

- System should display validation error messages for required fields
- Customer should not be created

Actual Result:

Customer added successfully and displayed in the list.

Status: Pass

Figure 6 illustrates the customer creation form where validation prevents submission of empty fields.

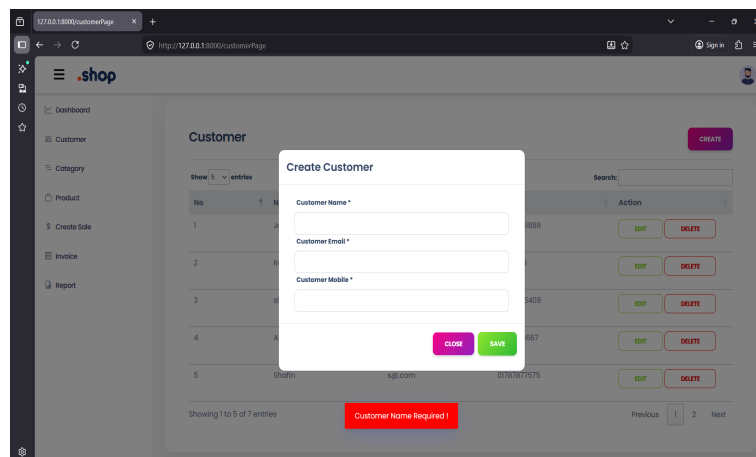


Figure 6: Required Fields Error Message

Test Case: Update Customer Information

Test Case ID: TC_CUST_03

Test Scenario: Update existing customer details

Pre-condition: User is on Customer page

Test Steps:

1. Click on EDIT button for a customer
2. Verify Customer ID is displayed
3. Enter updated value in Customer Name field
4. Enter updated value in Customer Email field
5. Enter updated value in Customer Mobile field
6. Click on UPDATE button

Test Data:

- Customer ID: 1
- Customer Name: Jon Doe
- Customer Email: jon12@gmail.com
- Customer Mobile: 01996553378

Expected Result:

- Customer information should be updated successfully
- Updated data should be reflected in the list

Actual Result:

Customer information updated successfully

Status: Pass

Figure 7 illustrates the customer update form where existing customer details can be modified.

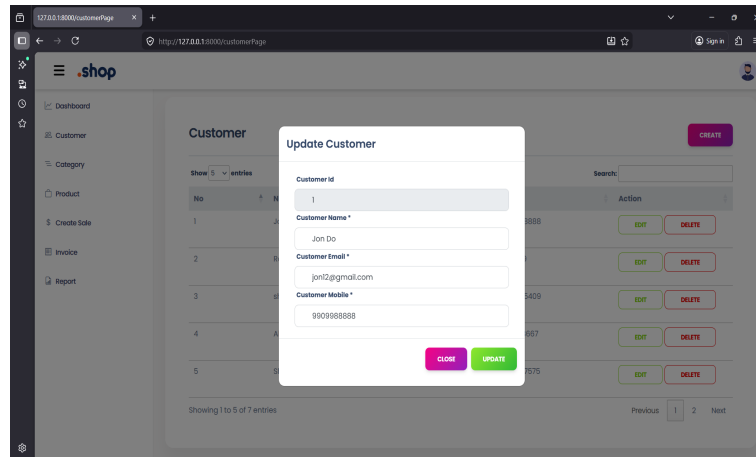


Figure 7: Customer Update Form Interface

Test Case: Delete Customer

Test Case ID: TC_CUST_04

Test Scenario: Delete an existing customer

Pre-condition: User is on Customer page with existing data

Test Steps:

1. Click on DELETE button for a customer
2. Click on the DELETE button to confirm the deletion

Expected Result:

- Customer should be removed from the list

Actual Result:

Customer deleted successfully

Status: Pass

Figure 8 shows the delete confirmation dialog that appears before removing a customer

from the system.

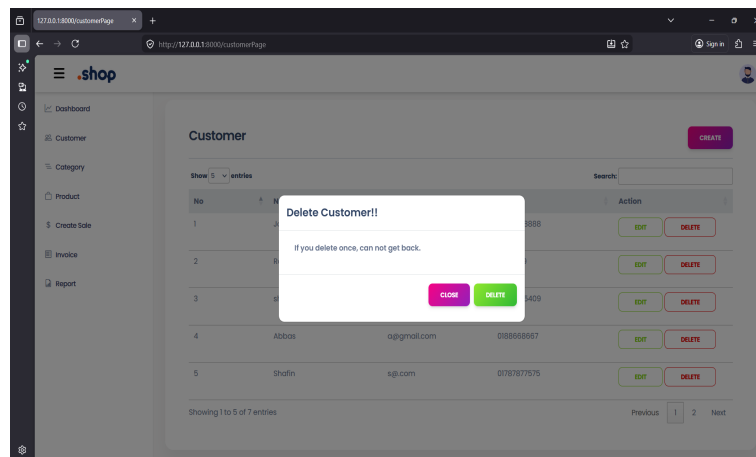


Figure 8: Customer Delete Confirmation Dialog

Testing Approach for Customer Module

The Customer module is tested using **CRUD Testing**, as it involves managing customer data through Create, Read, Update, and Delete operations.

The test cases cover adding new customers, updating existing customer information, deleting customers, and viewing customer lists. These operations confirm that the system correctly maintains customer data.

As the testing focuses on data operations rather than internal implementation, CRUD Testing is the most appropriate approach for this module.

Category Page

Test Case: Create Category with Valid Data

Test Case ID: TC_CAT_01

Test Scenario: Add a new category with valid name

Pre-condition: User is on Category page

Test Steps:

1. Click on CREATE button
2. Enter valid Category Name
3. Click on SAVE button

Test Data:

- Category Name: Beverages

Expected Result:

- Category should be successfully added
- New category should appear in the category list

Actual Result:

Category added successfully and displayed in the list

Status: Pass

Figure 9 illustrates the category creation form where users can input category name to add a new category.

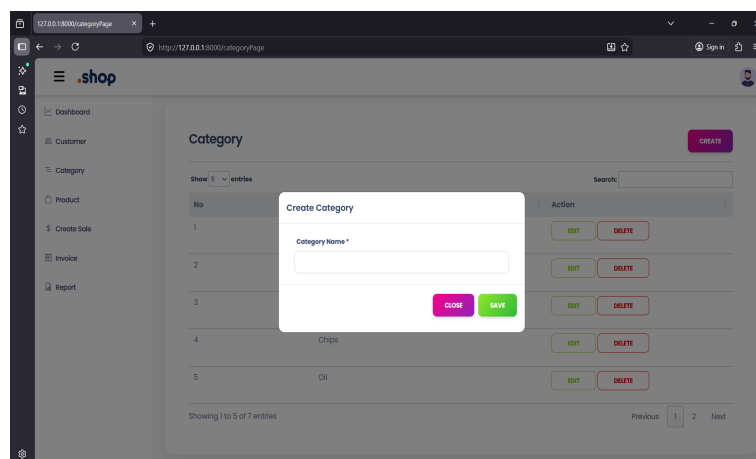


Figure 9: Category Creation Form Interface

Test Case: Create Category with Blank Name

Test Case ID: TC_CAT_02

Test Scenario: Add a new category without entering category name

Pre-condition: User is on Category page

Test Steps:

1. Click on CREATE button
2. Leave Category Name field empty
3. Click on SAVE button

Test Data:

- Category Name: blank

Expected Result:

- System should display validation error message
- Category should not be created

Actual Result:

Category added successfully and displayed in the list

Status: Fail

Figure 10 illustrates the category creation form where all input fields are left blank.

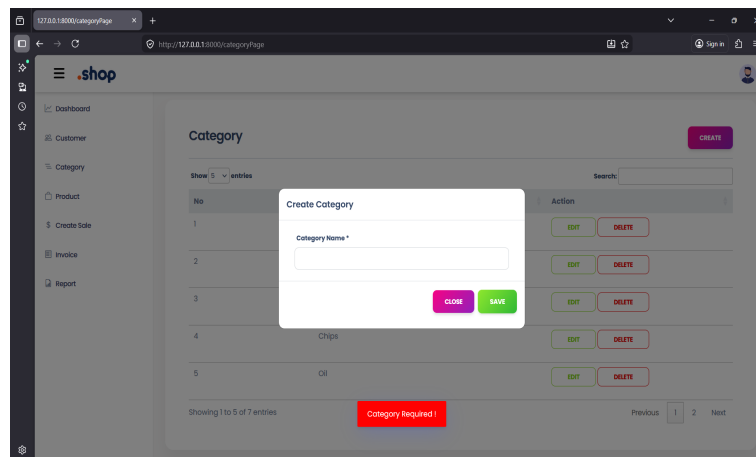


Figure 10: Category Creation Form with Blank Input

Test Case: Update Category Information

Test Case ID: TC_CAT_03

Test Scenario: Update existing category name

Pre-condition: User is on Category page with existing data

Test Steps:

1. Click on EDIT button for a category
2. Verify existing Category Name is displayed
3. Modify the Category Name.
4. Click on UPDATE button

Test Data:

- Category Name: Rice

Expected Result:

- Category name should be updated successfully
- Updated name should be reflected in the category list

Actual Result:

Category updated successfully and displayed in the list

Status: Pass

Figure 11 illustrates the category update form where existing category name can be modified.

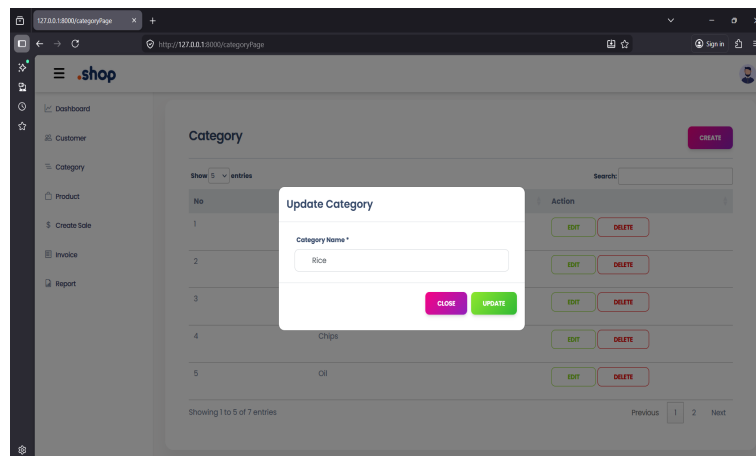


Figure 11: Category Update Form Interface

Test Case: Delete Category

Test Case ID: TC_CAT_04

Test Scenario: Delete an existing category

Pre-condition: User is on Category page with existing data

Test Steps:

1. Click on DELETE button for a category
2. Click on DELETE button to confirm

Test Data:

- Category ID: 1

Expected Result:

- Category should be removed from the list

Actual Result:

Category deleted successfully

Status: Pass

Figure 12 shows the delete confirmation dialog that appears before removing a category from the system.

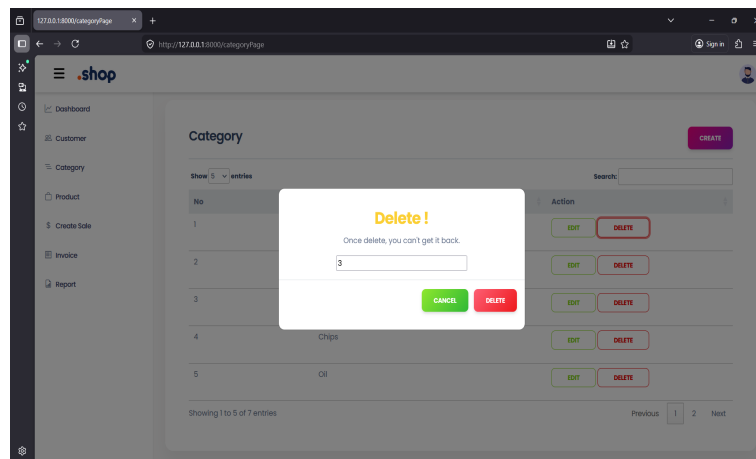


Figure 12: Category Delete Confirmation Dialog

Testing Approach for Category Module

The Category module is tested using **CRUD Testing**, as it primarily involves Create, Read, Update, and Delete operations on category data.

The test cases include adding a new category, updating existing category information, deleting a category, and viewing the category list. These operations ensure that the system handles data lifecycle management.

Since the focus of testing is on verifying data manipulation operations rather than internal logic, CRUD Testing is the most suitable approach for this module.

Product Page

Test Case: Create Product with Valid Data

Test Case ID: TC_PROD_01

Test Scenario: Add a new product with valid information

Pre-condition: User is on Product page

Test Steps:

1. Click on CREATE button
2. Select a Category
3. Enter Product Name
4. Enter Price
5. Enter Unit
6. Enter Quantity
7. Upload product image
8. Click on SAVE button

Test Data:

- Category: Fruits
- Name: Apple
- Price: 100
- Unit: kg
- Quantity: 10

- Image: apple.png

Expected Result:

- Product should be successfully created
- Product should appear in the product list with correct details

Actual Result:

Product created successfully and displayed in the list

Status: Pass

Figure 13 illustrates the product creation form where users can enter product details such as category, name, price, unit, quantity, and upload an image.

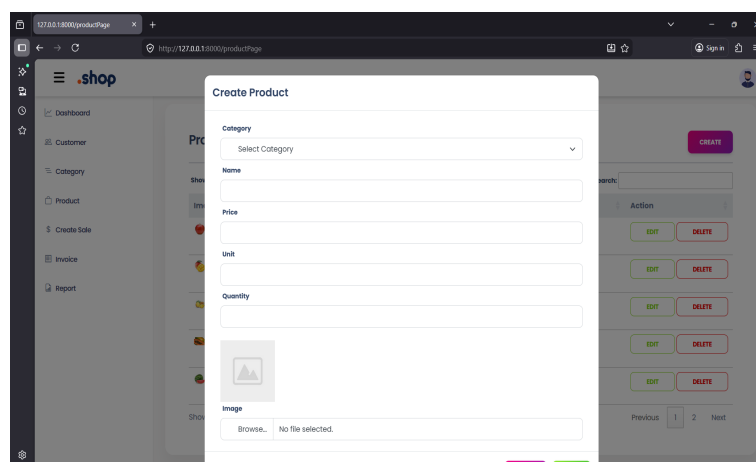


Figure 13: Product Creation Form Interface

Test Case: Create Product with Blank Data

Test Case ID: TC_PROD_02

Test Scenario: Attempt to add a new product without entering required information

Pre-condition: User is on Product page

Test Steps:

1. Click on CREATE button

2. Do not select any Category
3. Leave Product Name empty
4. Leave Price empty
5. Leave Unit empty
6. Leave Quantity empty
7. Do not upload product image
8. Click on SAVE button

Test Data:

- Category: blank
- Name: blank
- Price: blank
- Unit: blank
- Quantity: blank
- Image: blank

Expected Result:

- System should display validation error messages for required fields
- Product should not be created
- User should remain on the product creation page

Actual Result:

Product created successfully and displayed in the list

Status: Fail

Figure 14 illustrates the product creation form where validation errors appear when required fields are left blank.

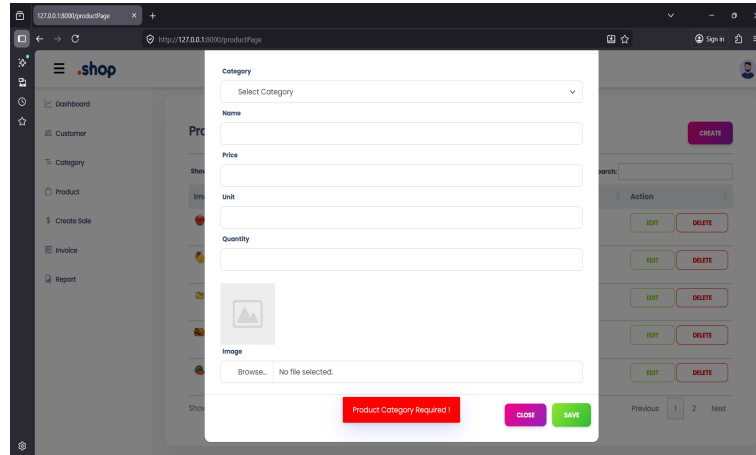


Figure 14: Product Creation Form with Validation Errors

Test Case: Update Product Information

Test Case ID: TC_PROD_03

Test Scenario: Update existing product details

Pre-condition: User is on Product page with existing product

Test Steps:

1. Click on EDIT button for a product
2. Verify Category is displayed
3. Update Product Name field
4. Update Price field
5. Update Unit field
6. Update Quantity field
7. Upload new image (optional)

8. Click on UPDATE button

Test Data:

- Category: Fruits
- Name: Apple
- Price: 120
- Unit: kg
- Quantity: 7

Expected Result:

- Product details should be updated successfully
- Updated data should be reflected in the product list

Actual Result:

Product updated successfully and changes reflected in the list

Status: Pass

Figure 15 illustrates the product update form where existing product details can be modified.

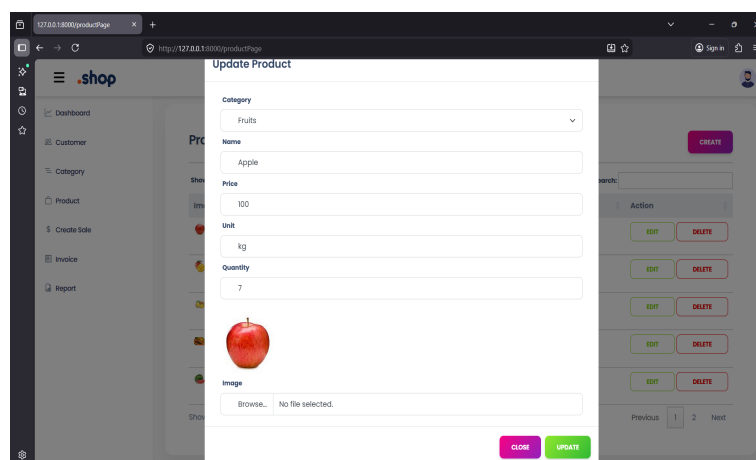


Figure 15: Product Update Form Interface

Test Case: Delete Product

Test Case ID: TC_PROD_04

Test Scenario: Delete an existing product

Pre-condition: User is on Product page with existing products

Test Steps:

1. Click on DELETE button for a selected product
2. In the confirmation dialog, click the **DELETE** button to confirm the deletion

Test Data:

- Product Name: Apple
- Category: Fruits

Expected Result:

- Product should be permanently removed
- Product should no longer appear in the product list

Actual Result:

Product deleted successfully and removed from the list

Status: Pass

Figure 16 shows the delete confirmation dialog that appears before removing a product from the system.

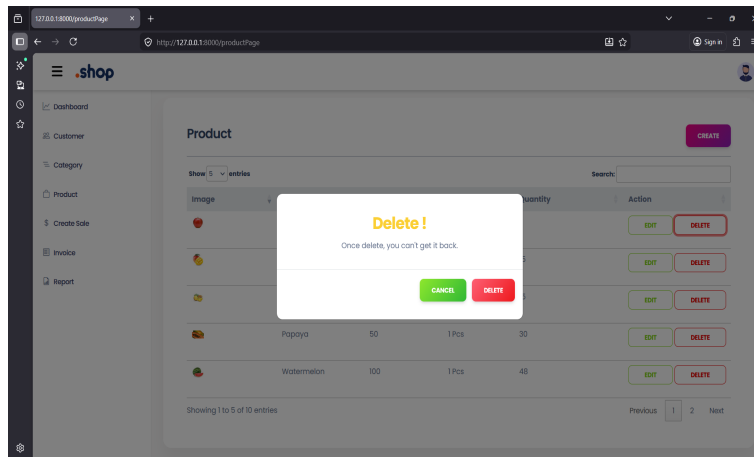


Figure 16: Product Delete Confirmation Dialog

Testing Approach for Product Module

The Product module is tested using **Validation Testing**, as the main objective is to verify whether the system properly validates user input before processing.

Test cases include creating products with valid data and attempting to submit forms with missing or invalid inputs such as empty fields. The results show that the system sometimes accepts invalid data, indicating validation issues.

Since the primary focus is on input validation and error handling, this module is best categorized under Validation Testing.

Create Sale Page

Test Case: Add Product to Sale

Test Case ID: TC_SALE_01

Test Scenario: Add a product to the sale list

Pre-condition: User is on Create Sale page

Test Steps:

1. Click on ADD button beside a product

Test Data:

- Product: Apple

Expected Result:

- Product should be added to invoice list
- Quantity and total should be displayed

Actual Result:

Product added successfully to the invoice list

Status: Pass

Figure 17 illustrates the Create Sale interface where users can add products and generate invoices.

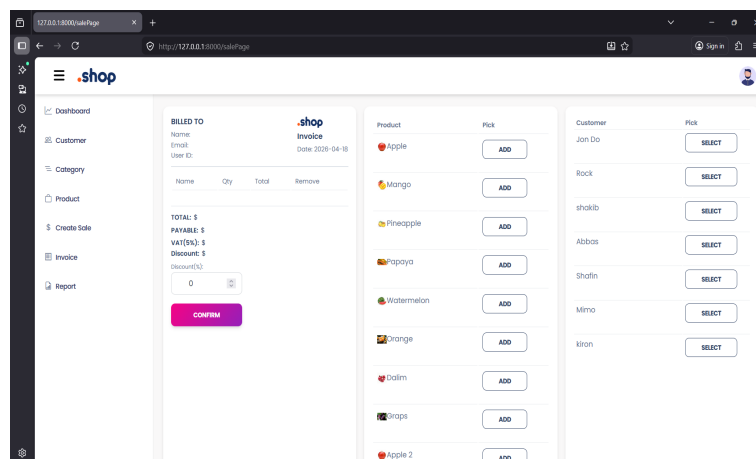


Figure 17: Create Sale Interface

Test Case: Select Customer

Test Case ID: TC_SALE_02

Test Scenario: Select a customer for the sale

Pre-condition: User is on Create Sale page

Test Steps:

1. Click on SELECT button beside a customer

Test Data:

- Customer: Jon Do

Expected Result:

- Customer details should appear in Billed To section

Actual Result:

Customer displayed correctly in Billed To section

Status: Pass

Test Case: Select Customer Without Choosing Any Customer

Test Case ID: TC_SALE_03

Test Scenario: Attempt to proceed without selecting any customer

Pre-condition: User is on Create Sale page

Test Steps:

1. Do not select any customer
2. Click on SELECT button

Test Data:

- Customer: Not selected

Expected Result:

- System should display an error message
- System should not allow proceeding without selecting a customer

Actual Result:

Customer displayed to Billed section

Status: Fail

Figure 18 shows the system behavior when no customer is selected and the error appears during the sale process.

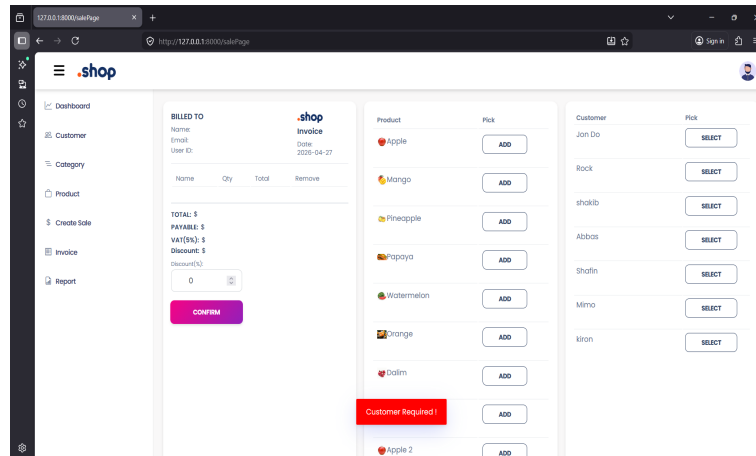


Figure 18: Customer Selection Error in Create Sale Page

Test Case: Apply Discount

Test Case ID: TC_SALE_04

Test Scenario: Apply discount on total amount

Pre-condition: Product already added to the sale

Test Steps:

1. Enter discount value
2. Click on CONFIRM button

Test Data:

- Discount: 10

Expected Result:

- Payable amount should decrease accordingly

Actual Result:

Discount applied correctly and payable amount reduced

Status: Pass

Test Case: Confirm Sale

Test Case ID: TC_SALE_05

Test Scenario: Complete the sale process

Pre-condition: Product added and customer selected

Test Steps:

1. Click on CONFIRM button

Test Data:

- Product: Apple
- Customer: Jon Do

Expected Result:

- Sale should be saved successfully
- Invoice should be generated

Actual Result:

Sale completed and invoice generated successfully

Status: Pass

Testing Approach for Sales Module

The Sales module is tested using **System Testing**, as it involves multiple components working together in a complete workflow.

The test cases include adding products to the sale, selecting customers, applying discounts,

and confirming the sale process. These steps verify that the entire sales workflow functions correctly from product selection to final transaction.

Since the testing evaluates the interaction between different parts of the system and ensures end-to-end functionality, it is categorized under System Testing.

Invoice Page

Test Case: Show Entries in Invoice List

Test Case ID: TC_INV_01

Test Scenario: Display number of invoice entries

Pre-condition: User is on Invoice page

Test Steps:

1. Select number of entries like (10, 25, 50) from dropdown

Test Data:

- Entries: 10

Expected Result:

- Selected number of invoices should be displayed

Actual Result:

Correct number of entries displayed

Status: Pass

Test Case: Search Invoice

Test Case ID: TC_INV_02

Test Scenario: Search invoice by customer name

Pre-condition: Invoice list available

Test Steps:

1. Enter "Rock" in search box

Test Data:

- Search: Rock

Expected Result:

- Matching invoices should be displayed

Actual Result:

Filtered invoice list displayed correctly

Status: Pass

Test Case: View Invoice Details

Test Case ID: TC_INV_03

Test Scenario: View detailed invoice

Pre-condition: Invoice exists

Test Steps:

1. Click on View (eye icon)

Test Data:

- Invoice ID: 15

Expected Result:

- Detailed invoice information should be displayed

Actual Result:

Invoice details displayed successfully

Status: Pass

Test Case: Delete Invoice

Test Case ID: TC_INV_04

Test Scenario: Delete an invoice

Pre-condition: Invoice exists

Test Steps:

1. Click on Delete icon
2. Confirm deletion

Test Data:

- Invoice ID: 15

Expected Result:

- Invoice should be removed from list

Actual Result:

Invoice deleted successfully

Status: Pass

Test Case: Navigate to Create Sale

Test Case ID: TC_INV_05

Test Scenario: Navigate to Create Sale page

Pre-condition: User is on Invoice page

Test Steps:

1. Click on CREATE SALE button

Test Data:

- None

Expected Result:

- User should be redirected to Create Sale page

Actual Result:

Navigation successful

Status: Pass

Figure 19 illustrates the invoice page interface where users can view invoice list, search records, control entries, and perform actions such as view and delete.

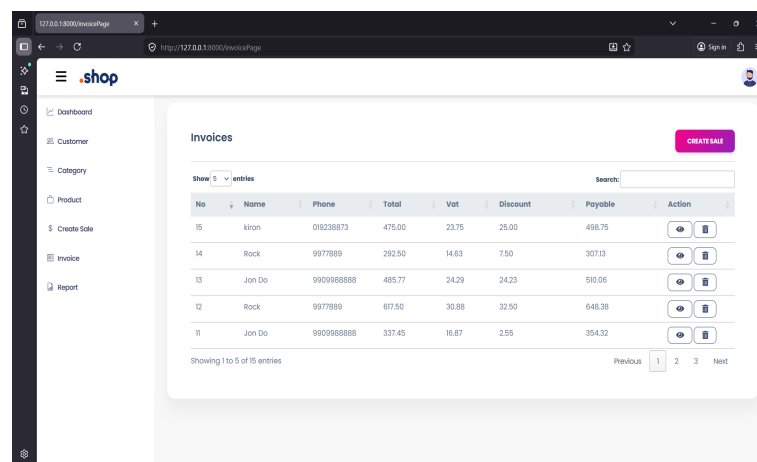


Figure 19: Invoice Page Interface

Testing Approach for Invoice Module

The Invoice module is tested using **Functional Testing**, as it focuses on verifying specific features of the system.

The test cases include viewing invoice entries, searching invoices, displaying detailed invoice information, deleting invoices, and navigating between pages. These tests ensure that each function operates according to the system requirements.

Since the testing checks individual functionalities without considering internal implementation, this module falls under Functional Testing.

Report Page

Test Case: Download Sales Report with Valid Dates

Test Case ID: TC_REPORT_01

Test Scenario: Download sales report by selecting Date From and Date To

Pre-condition: User is on Sales Report page

Test Steps:

1. Enter Date From
2. Enter Date To
3. Click on DOWNLOAD button

Test Data:

- Date From = 01/01/2026
- Date To = 01/31/2026

Expected Result:

- Report should be downloaded successfully based on selected date range

Actual Result:

Report downloaded successfully

Status: Pass

Figure 20 shows the sales report interface where users can select date range and download reports.

Test Case: Download Without Selecting Dates

Test Case ID: TC_REPORT_02

Test Scenario: Attempt to download report without selecting Date From and Date To

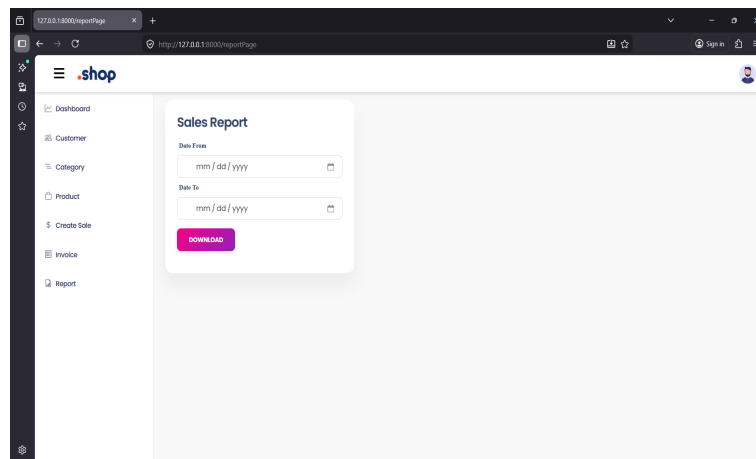


Figure 20: Sales Report Page

Pre-condition: User is on Sales Report page

Test Steps:

1. Leave Date From empty
2. Leave Date To empty
3. Click on DOWNLOAD button

Test Data:

- Date From = None
- Date To = None

Expected Result:

- System should display validation message

Actual Result:

Report downloaded without validation

Status: Fail

Figure 21 shows the sales report interface where a date range error is displayed when no data is provided.

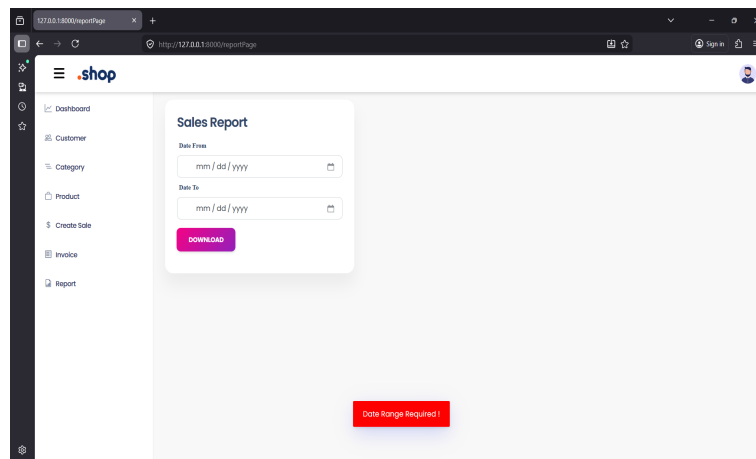


Figure 21: Sales Report Page

Testing Approach for Report Generation Module

The Report Generation module is tested using **Functional Testing**, as it focuses on verifying whether the system performs its intended functions correctly.

Test cases include generating reports with valid date ranges and attempting to download reports without selecting required inputs. The system behavior is checked against expected functionality.

Since the testing ensures that the system features work according to requirements, this module is categorized under Functional Testing.